КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМ. ІГОРЯ СІКОРСЬКОГО

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3

на тему: «Реалізація основних асиметричних криптосистем»

Виконали:
студентки групи ФІ-62 мн
Дегтярьова Марія
Тимошенко Марія

Київ – 2026

Зміст

1 Дослідження принципів асиметричної криптографії 2

2 Практичне дослідження цифрового підпису 5

3 Практичне дослідження узгодження спільного секрету 9

4 Вибір криптографічного механізму 12

5 Аналіз помилок застосування асиметричної криптографії 14

6 Сучасна постквантова асиметрична криптографія 16

7 Індивідуальне завдання 18

Висновки 19

Перелік використаних джерел 20

Мета роботи: дослідження принципів побудови та застосування сучасних асиметричних криптосистем, отримання практичних навичок використання механізмів цифрового підпису та узгодження ключів, аналіз особливостей безпечного застосування асиметричних криптографічних алгоритмів.

1 Дослідження принципів асиметричної криптографії

1.1 У чому полягає основний принцип асиметричної криптографії та які функції виконують відкритий і закритий ключі?

Основний принцип асиметричної криптографії полягає у використанні пари математично пов'язаних ключів: відкритого та закритого. Відкритий ключ можна вільно передавати іншим користувачам, тоді як закритий ключ повинен зберігатися в таємниці його власником.

Залежно від призначення криптографічної операції ключі виконують різні функції. Під час шифрування повідомлення використовується відкритий ключ одержувача, а для його розшифрування — відповідний закритий ключ. У випадку цифрового підпису, навпаки, закритий ключ використовується для створення підпису, а відкритий — для його перевірки. Таким чином, асиметрична криптографія дозволяє забезпечити конфіденційність інформації, автентичність її джерела та цілісність повідомлень [1].

1.2 Які принципові відмінності між асиметричним шифруванням, цифровим підписом та узгодженням спільного секрету (key agreement)?

Ці три механізми мають різне призначення.

Асиметричне шифрування використовується для забезпечення конфіденційності інформації. Відправник шифрує повідомлення відкритим ключем одержувача, а розшифрувати його може лише власник відповідного закритого ключа.

Цифровий підпис призначений для підтвердження автентичності відправника та цілісності повідомлення. Він створюється за допомогою закритого ключа відправника, а перевіряється за допомогою його відкритого ключа. Зміна повідомлення призводить до того, що перевірка підпису завершується невдало.

Узгодження спільного секрету дозволяє двом сторонам, які обмінюються відкритими даними, незалежно отримати однаковий секретний ключ. Надалі цей ключ можна використовувати для симетричного шифрування інформації.

Отже, основна відмінність полягає в результаті їх використання: шифрування захищає зміст повідомлення, цифровий підпис підтверджує його походження та цілісність, а узгодження ключів дозволяє отримати спільний секрет для подальшого захищеного обміну даними [1, 3].

1.3 Які задачі вирішують RSA, ECDSA/EdDSA та ECDH/X25519? Чому ці алгоритми не можна вважати взаємозамінними?

RSA, ECDSA, EdDSA та ECDH/X25519 є криптографічними алгоритмами з відкритим ключем, проте вони відрізняються за призначенням.

RSA — алгоритм, який може використовуватися як для асиметричного шифрування, так і для створення та перевірки цифрових підписів. Його стійкість ґрунтується на складності факторизації великих цілих чисел.

ECDSA та EdDSA призначені для створення і перевірки цифрових підписів на основі криптографії еліптичних кривих. ECDSA використовує випадкове значення під час створення кожного підпису, тоді як EdDSA використовує детермінований механізм формування підпису.

ECDH та X25519 використовуються для узгодження спільного секрету між двома сторонами. Вони дозволяють отримати однаковий секрет, не передаючи його безпосередньо мережею.

Ці алгоритми не є взаємозамінними, оскільки розв'язують різні криптографічні задачі. Наприклад, ECDSA та EdDSA забезпечують цифровий підпис, але не призначені для узгодження спільного секрету. Натомість ECDH та X25519 дозволяють узгодити секрет, але самі по собі не забезпечують цифрового підпису. RSA має ширший набір застосувань, однак це не означає, що він може безпосередньо замінити будь-який інший алгоритм без зміни протоколу та способу його використання [1, 2, 3].

1.4 Чому довжини ключів RSA та криптосистем на еліптичних кривих не можна безпосередньо порівнювати для оцінювання рівня криптографічної стійкості?

Безпосередньо порівнювати довжини ключів RSA та криптосистем на еліптичних кривих некоректно, оскільки їхня стійкість ґрунтується на різних математичних задачах. Для RSA основною є складність факторизації великого числа, а для криптосистем на еліптичних кривих — складність задачі дискретного логарифмування в групі точок еліптичної кривої.

Тому однакова кількість бітів у ключах не означає однаковий рівень захисту. Наприклад, ключ RSA довжиною 3072 біти та ключ на еліптичній кривій з порядком базової точки приблизно 256 біт забезпечують орієнтовно однаковий рівень класичної криптографічної стійкості — 128 біт.

Отже, для порівняння таких систем потрібно враховувати не лише довжину ключа, а й складність математичної задачі, на якій ґрунтується їхня безпека. Саме тому для оцінювання стійкості використовують поняття рівня криптографічної безпеки (security strength), а не просто кількість бітів у ключі [4].

1.5 Для чого при використанні RSA застосовуються спеціальні схеми доповнення (padding)? Які ризики створює неправильний вибір або використання padding?

Схема доповнення при використанні RSA застосовується для усунення математичних дефектів алгоритму та забезпечення необхідного рівня криптографічної стійкості. Застосування padding перетворює детерміноване шифрування на ймовірнісне за рахунок додавання випадкових байтів, що забезпечує формування нового шифротексту при кожному повторному шифруванні однакових даних [1]. Крім того, схеми доповнення руйнують гомоморфну мультиплікативну структуру шифру та захищають від атак добування кореня при малих значеннях публічної експоненти. Неправильний вибір або використання застарілих схем доповнення, таких як PKCS #1 v1.5, створює ризики реалізації атак на основі витоку інформації про помилки доповнення, відомих як атака Блейхенбахера або Padding Oracle. Для усунення цих вразливостей стандарт NIST SP 800-56B вимагає використання сучасних схем RSA-OAEP для шифрування та RSA-PSS для цифрових підписів [1, 7].

1.6 Яку роль виконує nonce при формуванні цифрового підпису ECDSA? Які наслідки може мати повторне використання одного nonce для різних повідомлень?

Під час створення цифрового підпису ECDSA значення nonce виконує роль одноразового випадкового числа, яке використовується для обчислення тимчасової точки на еліптичній кривій та формування першої компоненти підпису. Це значення має бути строго унікальним та непередбачуваним для кожного нового підпису [6]. Повторне використання одного й того самого значення nonce для підписання двох різних повідомлень на одному закритому ключі призводить до критичних наслідків у вигляді повного математичного розкриття приватного ключа підписувача. Атакуючий може обчислити різницю підписів, виявити значення secret nonce та на його основі легко відновити закритий ключ. Для запобігання цьому варто застосовувати детерміновану генерацію nonce або використовувати алгоритм EdDSA, який за своєю конструкцією усуває ризик повторного використання тимчасового параметра [6, 2, 5].

1.7 Чому асиметричне шифрування зазвичай не використовується безпосередньо для захисту великих обсягів даних? Що таке гібридна криптосистема?

Асиметричне шифрування не застосовується безпосередньо для захисту великих обсягів даних через низьку обчислювальну швидкість алгоритмів та жорсткі обмеження на розмір вхідного блоку. Операції піднесення до степеня за модулем великих чисел

потребують значно більше ресурсів процесора порівняно із симетричними блоковими шифрами [4]. Гібридна криптосистема є комбінованим криптографічним механізмом, який поєднує переваги асиметричної та симетричної криптографії. У такій системі сам обсяг даних шифрується швидким симетричним шифром, наприклад AES-256-GCM, за допомогою одноразового випадкового сеансового ключа. Сам сеансовий ключ має малий розмір і шифрується відкритим асиметричним ключем отримувача за допомогою RSA-OAEP або механізмів інкапсуляції ключів KEM [4, 1].

1.8 Які вимоги висуваються до зберігання та використання закритого ключа асиметричної криптосистеми?

Вимоги до зберігання та використання закритого ключа є забезпечення його конфіденційності та цілісності на всіх етапах життєвого циклу [4, 8]. Закритий ключ повинен зберігатися в ізольованому середовищі із застосуванням суворого розмежування прав доступу. При збереженні на носіях ключ обов'язково захищається симетричним шифруванням із використанням функцій виведення ключів на основі паролів, таких як PBKDF2 або Argon2id. Для систем із високим рівнем безпеки вимагається використання апаратних модулів безпеки HSM, TPM або смарт-карт, які унеможливлюють експорт ключа у відкритому вигляді за межі пристрою [8]. Під час використання в оперативній пам'яті ключ має захищатися від витоків через побічні канали, а після завершення операції підлягає негайному та безповоротному знищенню процедурою zeroization [4, 8].

2 Практичне дослідження цифрового підпису

2.1 Вибір криптографічного алгоритму та бібліотеки

Для практичного дослідження обрано алгоритм Ed25519, який належить до сімейства алгоритмів цифрового підпису на основі еліптичних кривих. Він забезпечує створення та перевірку цифрових підписів і характеризується високою швидкістю роботи та відносно невеликим розміром ключів.

Для реалізації використано бібліотеку `cryptography` мови Python, яка підтримує роботу з Ed25519. Вона надає готові засоби для генерації ключів, створення цифрового підпису та його перевірки, тому самостійна реалізація математичних операцій алгоритму не потрібна [2, 5].

2.2 Генерація пари ключів

На першому етапі генерується пара ключів: закритий ключ та відповідний йому відкритий ключ. Закритий ключ використовується для створення цифрового підпису, а відкритий — для його перевірки.

```
1 from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
```

```
from cryptography.exceptions import InvalidSignature

private_key = Ed25519PrivateKey.generate()
public_key = private_key.public_key()

print("1. Key pair generated.")
```

Результат виконання:

```
1. Key pair generated.
```

2.3 Створення тестового повідомлення

Для проведення експерименту створюється тестове повідомлення. Оскільки криптографічні операції виконуються над байтовими даними, текст перетворюється за допомогою методу `encode()`.

```
message = "This is a test message for digital signature verification."
message = message.encode("utf-8")

print("\n2. Test message:")
print(message.decode("utf-8"))
```

Результат виконання:

```
2. Test message:
This is a test message for digital signature verification.
```

2.4 Формування цифрового підпису

Для створення цифрового підпису використовується закритий ключ та підготовлене повідомлення. Результатом є послідовність байтів, яка надалі використовуватиметься для перевірки автентичності повідомлення.

```
signature = private_key.sign(message)

print("\n3. Digital signature created.")
print("Signature:", signature.hex())
```

Результат виконання:

```
3. Digital signature created.
Signature: 3c3a30bd02c73ba3daf57323007d6dcd828f2d51b1f0a01df2a63f90ab7d95d6236f0ea1c
9b9d5ab7aa5320ada4a30b3b68f57deb5c78f317b7e9fa546937904
```

Значення підпису залежить від згенерованої пари ключів, тому під час кожного запуску програми воно буде іншим.

2.5 Перевірка цифрового підпису

На цьому етапі виконується перевірка створеного підпису за допомогою відкритого ключа. Якщо підпис відповідає повідомленню та відкритому ключу, перевірка завершується успішно. Інакше бібліотека генерує виняток `InvalidSignature`.

```
print("\n4. Verification of the original message:")

try:
    public_key.verify(signature, message)
    print("Signature is valid.")
except InvalidSignature:
    print("Signature is invalid.")
```

Результат виконання:

```
4. Verification of the original message:
Signature is valid.
```

2.6 Перевірка підпису після зміни повідомлення

Для перевірки цілісності повідомлення змінюється його вміст, після чого виконується повторна перевірка початкового цифрового підпису. Очікується, що перевірка завершиться невдало, оскільки підпис був створений для іншого повідомлення.

```
modified_message = "This is a modified test message.".encode("utf-8")

print("\n5. Verification of the modified message:")

try:
    public_key.verify(signature, modified_message)
    print("Signature is valid.")
except InvalidSignature:
    print("Signature is invalid: message was modified.")
```

Результат виконання:

```
5. Verification of the modified message:
Signature is invalid: message was modified.
```

2.7 Перевірка підпису з використанням іншого відкритого ключа

На останньому етапі генерується інша пара ключів. Для перевірки початкового підпису використовується новий відкритий ключ, який не відповідає закритому ключу, за допомогою якого було створено підпис.

```
other_private_key = Ed25519PrivateKey.generate()
other_public_key = other_private_key.public_key()

print("\n6. Verification with another public key:")

try:
    other_public_key.verify(signature, message)
    print("Signature is valid.")
except InvalidSignature:
    print("Signature is invalid: another public key was used.")
```

Результат виконання:

```
6. Verification with another public key:
Signature is invalid: another public key was used.
```

2.8 Аналіз результатів

У результаті експерименту було реалізовано повний цикл роботи з цифровим підписом за допомогою алгоритму Ed25519.

Під час перевірки початкового повідомлення з використанням відповідного відкритого ключа підпис було успішно підтверджено. Після зміни повідомлення перевірка завершилася невдало, оскільки цифровий підпис не відповідає зміненим даним. Аналогічно, використання іншого відкритого ключа також призвело до невдалої перевірки.

Отримані результати демонструють основні властивості цифрового підпису:

1. Цілісність — дозволяє виявити зміну повідомлення після створення підпису.
2. Автентичність — дає змогу перевірити, чи відповідає підпис закритому ключу, пов'язаному з використаним відкритим ключем.
3. Невідмовність — за умови надійного зберігання закритого ключа та підтвердження його належності конкретному власнику, підпис може використовуватися як доказ авторства повідомлення.

Таким чином, експеримент підтвердив, що цифровий підпис дозволяє контролювати цілісність повідомлень та перевіряти їхнє походження. При цьому він не забезпечує конфіденційності, оскільки саме повідомлення залишається доступним для читання.

3 Практичне дослідження узгодження спільного секрету

3.1 Вибір криптографічного механізму та бібліотеки

Для проведення експерименту з узгодження спільного секрету (Key Agreement) обрано механізм X25519, який базується на еліптичній кривій Curve25519. Цей протокол є сучасним стандартом (RFC 7748), оскільки забезпечує високу швидкодію, стійкість до атак по побічних каналах та високий рівень безпеки (128 біт).

Для реалізації використано бібліотеку `cryptography` мови Python, зокрема модуль `cryptography.hazmat.primitives.asymmetric.x25519`.

3.2 Генерація ключів Alice

Аліса генерує власну пару ключів X25519 (приватний та відповідний йому відкритий ключ).

```
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives import serialization

alice_private_key = x25519.X25519PrivateKey.generate()
alice_public_key = alice_private_key.public_key()

print("Alice key pair generated")
```

Результат виконання:

```
Alice key pair generated
```

3.3 Генерація ключів Bob

Боб незалежно генерує власну пару ключів X25519.

```
bob_private_key = x25519.X25519PrivateKey.generate()
bob_public_key = bob_private_key.public_key()

print("Bob key pair generated")
```

Результат виконання:

```
Bob key pair generated
```

3.4 Обмін відкритими ключами

Учасники експортують свої відкриті ключі у байтовий формат для подальшої передачі через недовірену мережу.

```
alice_pub_bytes = alice_public_key.public_bytes(
    encoding=serialization.Encoding.Raw,
    format=serialization.PublicFormat.Raw
)
bob_pub_bytes = bob_public_key.public_bytes(
    encoding=serialization.Encoding.Raw,
    format=serialization.PublicFormat.Raw
)

print("\nPublic keys exchanged via network:")
print("Alice Public Key (hex):", alice_pub_bytes.hex())
print("Bob Public Key (hex): ", bob_pub_bytes.hex())
```

Результат виконання:

```
Public keys exchanged via network:
Alice Public Key (hex): 8
    b1a30d12461b22fa4b51740dd7edee7ee9fbfa205896c4acfce331e551e3775
Bob Public Key (hex):
    d56e6bc0a7c316ad64557ccdfc04d86bdbac162c8e427972c47d0561be703805
```

3.5 Формування спільного секрету Alice

Аліса обчислює спільний секрет, використовуючи власний приватний ключ та відкритий ключ Боба.

```
alice_shared_secret = alice_private_key.exchange(bob_public_key)

print("\nAlice derived shared secret:")
print("Alice Secret (hex):", alice_shared_secret.hex())
```

Результат виконання:

```
Alice derived shared secret:
Alice Secret (hex): c75a40118b93810d7cfbcdb15267733444be22e913bd60e025c1146ae9b65665
```

3.6 Формування спільного секрету Bob

Боб обчислює спільний секрет, використовуючи власний приватний ключ та відкритий ключ Аліси.

```
bob_shared_secret = bob_private_key.exchange(alice_public_key)

print("\nBob derived shared secret:")
print("Bob Secret (hex): ", bob_shared_secret.hex())
```

Результат виконання:

```
Bob derived shared secret:
Bob Secret (hex): c75a40118b93810d7cfbcdb15267733444be22e913bd60e025c1146ae9b65665
```

3.7 Порівняння отриманих спільних секретів

Перевіряємо ідентичність обчислених секретів обома сторонами.

```
is_equal = (alice_shared_secret == bob_shared_secret)

print("\nComparison result:")
print("Shared secrets are equal:", is_equal)
```

Результат виконання:

```
Comparison result:
Shared secrets are equal: True
```

3.8 Аналіз результатів

Чому Alice та Bob отримали однаковий спільний секрет?

Аліса та Боб отримали однаковий секрет завдяки математичним властивостям операцій над точками еліптичної кривої. Обчислення базується на комутативності скалярного множення:

$$K_{shared} = d_A \cdot (d_B \cdot G) = d_B \cdot (d_A \cdot G) = (d_A \cdot d_B) \cdot G$$

де d_A та d_B — приватні ключі Аліси та Боба, а G — базова точка кривої X25519.

Які дані передавалися між сторонами?

Між сторонами через мережу передавалися виключно відкриті ключі (байтові послідовності $A_{pub} = d_A \cdot G$ та $B_{pub} = d_B \cdot G$).

Чи передавався сам спільний секрет через мережу?

Ні, сам спільний секрет через мережу ніколи не передавався. Обидві сторони обчислили його повністю незалежно та локально у власному середовищі.

Чи може пасивний спостерігач сформувати цей секрет, маючи лише передані відкриті ключі?

Ні, пасивний спостерігач не може сформувати секрет. Для цього йому потрібно знати хоча б один із приватних ключів (d_A або d_B). Знаходячись у мережі та бачачи лише A_{pub} і B_{pub}, визначити значення $d_A \cdot d_B \cdot G$ є обчислювально нездійсненною задачею, яка називається обчислювальною задачею Діффі–Хеллмана (CDH) і спирається на складність дискретного логарифмування в групі точок еліптичної кривої.

Чому сам по собі механізм узгодження ключа не обов'язково підтверджує особу іншої сторони?

Базовий протокол ECDH є анонімним і не забезпечує аутентифікацію сторін за замовчуванням. Відкритий ключ X25519 сам по собі не містить інформації про особу власника. Це робить протокол вразливим до атак типу «людина посередині» (Man-in-the-Middle), коли зловмисник може перехопити та підмінити відкриті ключі під час обміну. Для підтвердження особи відкриті ключі повинні додатково підписуватися цифровим підписом або підтверджуватися сертифікатами X.509 у рамках інфраструктури відкритих ключів (PKI) [3, 4].

4 Вибір криптографічного механізму

4.1 Ситуація 1. Захист електронного документа

Умова: Необхідно забезпечити можливість перевірити, що електронний документ був створений певним власником закритого ключа та не був модифікований після підписання.

Запропонований криптографічний механізм: Цифровий підпис та хешування.

Можливий алгоритм або схема: Ed25519.

Обґрунтування вибору: Спочатку для документа обчислюється хеш, після чого він підписується приватним ключем власника. Для перевірки використовується відповідний відкритий ключ. Якщо документ було змінено після підписання, перевірка підпису завершиться невдало. Ed25519 забезпечує високу швидкість роботи та компактний розмір підпису. При цьому для підтвердження особи власника необхідно також мати надійний зв'язок між його особою та відкритим ключем.

4.2 Ситуація 2. Формування спільного секретного ключа

Умова: Alice та Bob повинні сформувати спільний секретний ключ, обмінюючись інформацією через недовірену мережу.

Запропонований криптографічний механізм: Узгодження ключів.

Можливий алгоритм або схема: ECDH на основі еліптичної кривої X25519.

Обґрунтування вибору: Кожна сторона генерує власну пару ключів і обмінюється відкритими ключами. Після цього Alice та Bob незалежно обчислюють однаковий спільний секрет, використовуючи власний приватний ключ та відкритий ключ іншої сторони. Сам секрет через мережу не передається. X25519 забезпечує ефективне узгодження ключів, однак для захисту від атак типу «людина посередині» необхідна автентифікація сторін.

4.3 Ситуація 3. Конфіденційна передача великого файла

Умова: Необхідно передати файл розміром 2 ГБ у конфіденційному вигляді. Відкритий ключ отримувача відомий відправнику.

Запропонований криптографічний механізм: Гібридне шифрування, яке поєднує асиметричну криптографію та симетричне шифрування.

Можливий алгоритм або схема: Для встановлення секретного ключа можна використати RSA-OAEP, а для шифрування файла — AES-256-GCM.

Обґрунтування вибору та переваги гібридної схеми: Спочатку відправник генерує випадковий симетричний ключ, яким шифрує файл за допомогою AES-256-GCM. Сам симетричний ключ шифрується відкритим ключем отримувача за допомогою RSA-OAEP. Отримувач розшифровує ключ своїм приватним ключем, а потім використовує його для відновлення початкового файла. Безпосереднє асиметричне шифрування всього файла не є оптимальним, оскільки такі алгоритми мають обмеження на розмір даних, які можна зашифрувати за одну операцію, та працюють повільніше за симетричні. Гібридна схема дозволяє швидко зашифрувати великий обсяг інформації, використовуючи асиметричний алгоритм лише для захисту невеликого симетричного ключа.

4.4 Ситуація 4. Безпечне зберігання паролів

Умова: Сервер повинен безпечно зберігати паролі користувачів та перевіряти їх під час автентифікації.

Чи потрібна асиметрична криптографія: Ні, для цього завдання асиметрична криптографія не потрібна. Паролі не потрібно шифрувати для подальшого відновлення, достатньо зберігати їхні спеціально обчислені хеші.

Запропонований механізм: Хешування паролів із використанням індивідуальної випадкової солі.

Обґрунтування вибору: Можна використати алгоритм Argon2id, який спеціально призначений для захищеного зберігання паролів. Під час реєстрації для кожного пароля генерується випадкова сіль, після чого обчислюється його хеш. На сервері зберігаються сіль та отриманий хеш, але не сам пароль. Під час автентифікації сервер повторно обчислює хеш введеного пароля з тією самою сіллю та порівнює результати. Argon2id ускладнює масовий підбір паролів завдяки використанню обчислювальних ресурсів і пам'яті.

4.5 Ситуація 5. Автентичність оновлень програмного забезпечення

Умова: Виробник випускає оновлення програмного забезпечення для великої кількості пристроїв. Створювати справжні оновлення може лише виробник, а кожен пристрій повинен мати можливість перевірити їх автентичність.

Запропонований криптографічний механізм: Цифровий підпис та хешування.

Можливий алгоритм або схема: Ed25519.

Розподіл ключів між виробником та пристроями: Виробник зберігає приватний ключ у захищеному середовищі та використовує його для підписання оновлень. На кожному пристрої зберігається відповідний відкритий ключ, який використовується для перевірки підпису. Приватний ключ виробника ніколи не передається пристроям.

Обґрунтування вибору: Перед розповсюдженням оновлення виробник створює йо-

го цифровий підпис. Пристрій після отримання оновлення перевіряє підпис за допомогою відкритого ключа. Якщо перевірка успішна, це підтверджує, що оновлення було підписане власником відповідного приватного ключа та не було змінене після підписання. Такий механізм дозволяє поширювати оновлення серед великої кількості пристроїв без передавання їм секретного ключа.

5 Аналіз помилок застосування асиметричної криптографії

5.1 Ситуація 1. Неправильне використання padding у RSA

Умова: RSA використовується без рекомендованої схеми padding або із застарілою схемою, непридатною для відповідного сценарію.

Помилка: Використання сирого (textbook) RSA або застарілої схеми PKCS#1 v1.5 замість сучасних схем доповнення RSA-OAEP / RSA-PSS.

Можливі наслідки: Вразливість системи до атак типу Padding Oracle (атака Блейхенбахера), можливість підробки цифрових підписів, детермінованість шифротексту та ризик розшифрування повідомлень малої довжини через добування кореня.

Спосіб усунення: Обов'язкове переведення системи на стандарт RSA-OAEP для шифрування та RSA-PSS для цифрових підписів відповідно до вимог RFC 8017 [1].

5.2 Ситуація 2. Повторне використання nonce в ECDSA

Умова: Під час створення двох різних ECDSA-підписів було повторно використано однаковий nonce.

Помилка: Використання однакового або передбачуваного тимчасового параметра k (nonce) під час створення кількох підписів ECDSA.

Можливі наслідки: Повне математичне виведення та розкриття приватного ключа підписувача на основі двох відомих підписів і відповідних повідомлень. Повна компрометація системи.

Спосіб усунення: Застосування алгоритмів із детермінованою генерацією k на основі хешу повідомлення та приватного ключа (стандарт RFC 6979) або перехід на алгоритм EdDSA (Ed25519) [6, 2].

5.3 Ситуація 3. Передбачуваний генератор випадкових чисел

Умова: Криптографічні ключі генеруються з використанням передбачуваного генератора випадкових чисел.

Помилка: Застосування некриптографічних генераторів псевдовипадкових чисел (наприклад, `random.randint()` у Python) замість криптографічно стійких CSPRNG.

Можливі наслідки: Значне звуження простору ключового матеріалу, можливість

відтворення послідовності генератора зловмисником та швидкий повний перебір (brute-force) створених ключів.

Спосіб усунення: Використання виключно криптографічно стійких генераторів випадкових чисел (CSPRNG), підкріплених джерелами ентропії операційної системи (`os.urandom()`, `secrets`, системні виклики `getrandom()`) [4].

5.4 Ситуація 4. Відсутність перевірки відкритого ключа

Умова: Система отримує відкритий ключ від іншої сторони та використовує його без перевірки його належності очікуваному учаснику взаємодії.

Помилка: Нехтування автентифікацією відкритого ключа та відсутність перевірки цифрових сертифікатів партнерів по взаємодії.

Можливі наслідки: Успішна реалізація атаки «людина посередині» (Man-in-the-Middle). Зловмисник підміняє відкритий ключ власним, що дозволяє йому перехоплювати, розшифровувати та модифікувати весь трафік.

Спосіб усунення: Впровадження інфраструктури відкритих ключів (PKI), перевірка ланцюжків сертифікатів X.509 або використання механізмів фіксації ключів (Key Pinning) [4].

5.5 Ситуація 5. Повторне використання ключової пари

Умова: Одна й та сама пара ключів використовується одночасно для різних криптографічних призначений без аналізу сумісності таких способів використання.

Помилка: Мультифункціональне використання однієї пари ключів (наприклад, одночасне застосування однієї RSA-пари для шифрування даних і для накладання цифрових підписів).

Можливі наслідки: Перехресні атаки між різними протоколами. Зловмисник може змусити систему підписати зашифроване повідомлення, перетворюючи сервіс підпису на оракул розшифрування.

Спосіб усунення: Суворе дотримання принципу розділення ключів (Key Separation) шляхом випуску окремих пар ключів для кожної криптографічної операції та контролю прапорців призначення в сертифікатах (`KeyUsage`) [4].

5.6 Загальний висновок щодо помилок застосування

Наведені ситуації демонструють, що математична стійкість криптографічного алгоритму сама по собі негарантує безпечності інформаційної системи. Більшість критичних вразливостей виникає не через злам математичної основи шифрів, а внаслідок помилок їх реалізації та архітектурного застосування: використання передбачуваних джерел випадковості, некоректного вибору схем доповнення (padding), повторного використання тимчасових параметрів (nonce), відсутності автентифікації відкритих ключів та порушення

принципу розділення ключів. Для забезпечення реальної безпеки необхідно чітко дотримуватись протоколів та вимог міжнародних криптографічних стандартів.

6 Сучасна постквантова асиметрична криптографія

6.1 Чому поява достатньо потужного квантового комп'ютера створює загрозу для RSA та криптосистем на еліптичних кривих?

Безпека RSA та криптосистем на еліптичних кривих ґрунтується на математичних задачах, які складно розв'язати за допомогою звичайних комп'ютерів. Для RSA такою задачею є факторизація великих цілих чисел, а для криптосистем на еліптичних кривих— задача дискретного логарифмування.

Поява достатньо потужного квантового комп'ютера змінює цю ситуацію. Алгоритм Шора дозволяє розв'язувати обидві задачі значно ефективніше, ніж класичні алгоритми. Тому квантовий комп'ютер, який матиме достатню кількість стабільних кубітів, зможе отримувати секретні ключі на основі відкритих даних. Це поставить під загрозу конфіденційність інформації та надійність цифрових підписів, створених за допомогою цих алгоритмів.

6.2 Пояснити поняття постквантової криптографії (Post-Quantum Cryptography, PQC).

Постквантова криптографія — це напрям криптографії, який розробляє алгоритми, стійкі як до звичайних, так і до квантових комп'ютерів. Її мета — забезпечити захист інформації навіть тоді, коли з'являться достатньо потужні квантові обчислювальні системи.

Важливо, що такі алгоритми не потребують квантового комп'ютера для своєї роботи. Вони виконуються на звичайних комп'ютерах, але використовують математичні задачі, для яких наразі не відомо ефективних способів розв'язання навіть за допомогою квантових алгоритмів.

6.3 Визначити призначення ML-KEM та ML-DSA і пояснити, які традиційні криптографічні задачі вони повинні виконувати.

ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) — це механізм інкапсуляції ключів, призначений для безпечного встановлення спільного секретного ключа між двома сторонами. За своїм призначенням він виконує задачу, подібну до традиційних механізмів обміну ключами, наприклад Diffie–Hellman.

ML-DSA (Module-Lattice-Based Digital Signature Algorithm) — це алгоритм цифрового підпису, який дозволяє підтвердити справжність повідомлення та перевірити, чи не було його змінено. Таким чином, він виконує ті самі основні функції, що й традиційні алгоритми цифрового підпису, наприклад RSA або ECDSA.

Обидва алгоритми належать до постквантової криптографії та ґрунтуються на математичних задачах, пов'язаних із ґратками.

6.4 Пояснити різницю між механізмом встановлення ключа (KEM) та алгоритмом цифрового підпису.

KEM та цифровий підпис призначені для розв'язання різних криптографічних задач. Механізм встановлення ключа використовується для отримання спільного секрету між двома сторонами. Надалі цей секрет можна застосувати для симетричного шифрування даних.

Цифровий підпис, навпаки, не призначений для створення спільного ключа або приховування інформації. Його використовують для перевірки цілісності повідомлення та підтвердження того, що воно було підписане власником відповідного приватного ключа.

Отже, KEM забезпечує встановлення секретного ключа, а цифровий підпис — автентичність і цілісність даних.

6.5 Пояснити принцип гібридного встановлення ключів, при якому одночасно використовуються традиційний та постквантовий механізми.

Гібридне встановлення ключів передбачає одночасне використання двох механізмів: традиційного, наприклад ECDH, та постквантового, наприклад ML-KEM. Кожен із них формує власний спільний секрет, після чого ці секрети об'єднуються за допомогою спеціальної функції виведення ключів.

У результаті отримується спільний ключ, який використовується для подальшого шифрування даних. Такий підхід дозволяє поєднати перевірену часом класичну криптографію з новими постквантовими алгоритмами. Якщо один із механізмів виявиться вразливим, інший за умови правильної реалізації може забезпечити додатковий рівень захисту.

6.6 Пояснити, чому перехід інформаційних систем на постквантові алгоритми може відбуватися поступово.

Перехід на постквантову криптографію не можна виконати миттєво, оскільки сучасні інформаційні системи використовують велику кількість різних криптографічних алгоритмів, протоколів і програмних засобів. Їх заміна потребує часу, тестування та перевірки сумісності з наявним обладнанням і програмним забезпеченням.

Крім того, постквантові алгоритми можуть мати інші вимоги до розміру ключів, підписів і обсягу переданих даних. Тому їх впровадження може вимагати змін у роботі окремих систем. Поступовий перехід дозволяє спочатку використовувати гібридні рішення, перевірити їхню надійність і лише після цього повністю перейти на нові алгоритми.

7 Індивідуальне завдання

В межах індивідуального завдання за варіантом 1 досліджено асиметричний криптографічний механізм ElGamal.

7.1 Призначення та задача

Криптосистема ElGamal є асиметричним криптографічним алгоритмом, який було розроблено Тахером Ель Гамалем у 1985 році. Вона призначена для розв'язання двох основних задач захисту інформації: забезпечення конфіденційності даних шляхом асиметричного шифрування та забезпечення автентичності й цілісності повідомлень за допомогою цифрового підпису.

7.2 Основна ідея роботи

Основна ідея криптосистеми ElGamal базується на використанні односторонньої функції з таємним входом у мультиплікативній групі скінченного поля. Безпека системи повністю спирається на обчислювальну складність задачі дискретного логарифмування. Під час шифрування відправник використовує відкритий ключ одержувача та генерує одноразовий випадковий сеансовий параметр. Це забезпечує властивість ймовірнісного шифрування, коли один і той самий відкритий текст при кожному повторному шифруванні дає новий шифротекст, що унеможливлює атаки на основі аналізу повторюваності даних.

7.3 Основні криптографічні примітиви

В основі функціонування криптосистеми ElGamal лежать операції в мультиплікативних групах скінченних полів за модулем великого простого числа. Алгоритм спирається на складність обчислення дискретного логарифма у скінченному полі, застосовує односторонні криптографічні хеш-функції для формування цифрового підпису та вимагає використання криптографічно стійких генераторів псевдовипадкових чисел для вибору одноразового сеансового параметра.

7.4 Властивості безпеки

Криптосистема ElGamal забезпечує конфіденційність інформації, оскільки розшифрувати повідомлення може лише власник відповідного приватного ключа. Завдяки ймовірнісному шифруванню алгоритм гарантує непередбачуваність шифротексту. У режимі цифрового підпису система забезпечує автентичність джерела та невідмовність від авторства, а також контролює цілісність даних, унеможливлюючи їх непомітну модифікацію під час передачі.

7.5 Приклад практичного сценарію застосування

Математичні принципи ElGamal лежать в основі стандарту цифрового підпису DSA, а також застосовуються у протоколах захисту електронної пошти OpenPGP та GnuPG для асиметричного шифрування сеансових ключів і підписання повідомлень. Крім того, гомоморфні властивості шифру ElGamal дозволяють виконувати обчислення над зашифрованими даними без їх розшифрування, що широко використовується у протоколах електронного голосування для підрахунку голосів.

7.6 Основні обмеження та ризики практичного використання

Основним недоліком ElGamal є надмірність шифротексту, розмір якого удвічі перевищує розмір початкового відкритого тексту. Повторне використання одного й того самого випадкового сеансового параметра для шифрування різних повідомлень призводить до розкриття відкритого тексту. Без використання спеціальних схем доповнення ElGamal є податливим до атак на основі мультиплікативної гомоморфності, а самі модульні обчислення вимагають значно більше ресурсів порівняно із сучасними алгоритмами на еліптичних кривих.

Висновки

Під час виконання лабораторної роботи ми дослідили теоретичні та практичні засади функціонування сучасних асиметричних криптосистем, механізми формування цифрових підписів, узгодження спільного секрету, аналіз типових помилок їх застосування, а також ознайомилися з перспективними постквантовими стандартами та специфікою асиметричної криптосистеми ElGamal.

Аналізуючи теоретичні засади асиметричної криптографії, ми з'ясували, що відкритий та закритий ключі виконують чітко розмежовані функції залежно від контексту криптографічної операції: шифрування захищає конфіденційність, цифровий підпис гарантує автентичність джерела та цілісність даних, а протоколи узгодження ключів забезпечують формування спільного секрету через недовірені канали зв'язку. Дослідження критерію криптографічної стійкості підтвердило, що безпосередньо порівнювати довжини ключів RSA та еліптичних кривих некоректно, оскільки їх безпека спирається на принципово різні за складністю математичні задачі — факторизацію великих цілих чисел та дискретне логарифмування відповідно.

Практична частина роботи дозволила успішно реалізувати та протесувати алгоритм цифрового підпису Ed25519 та протокол узгодження ключів X25519 за допомогою бібліотеки cryptography мови Python. Експеримент з Ed25519 наочно підтвердив ключові властивості цифрового підпису: підпис є чутливим до найменших змін у повідомленні та робить перевірку недійсною (виняток InvalidSignature) у разі модифікації даних або використання стороннього відкритого ключа. Дослідження протоколу X25519 показово проде-

монструвало, як дві сторони (Alice та Bob), обмінюючись виключно відкритими ключами через недовірену мережу, незалежно й локально обчислюють ідентичний спільний секрет без його прямої передачі мережею.

Аналіз практичних ситуацій вибору криптографічних механізмів та аналіз типових помилок застосування довели, що математична стійкість алгоритму сама по собі не гарантує безпеки всієї системи. Критичні вразливості часто виникають на рівні реалізації та архітектури: використання некриптографічних генераторів випадкових чисел, некоректний вибір схем доповнення (padding) в RSA, повторне використання тимчасового параметра (nonce) в ECDSA, відсутність автентифікації відкритих ключів (ризик атак Man-in-the-Middle) або порушення принципу розділення ключів.

Розглядаючи напрям постквантової криптографії, ми з'ясували необхідність поступового переходу на нові стандарти на основі ґраток (ML-KEM та ML-DSA) через потенційну загрозу алгоритму Шора на квантових комп'ютерах, а в межах індивідуального завдання за варіантом 1 розкрили специфіку криптосистеми ElGamal, її ймовірнісні властивості шифрування та гомоморфні особливості, які знаходять застосування у протоколах електронного голосування та стандартах DSA/OpenPGP.

Перелік використаних джерел

Література

[1] Moriarty, K., Kaliski, B., Jonsson, J., Rusch, A. *PKCS #1: RSA Cryptography Specifications Version 2.2*. RFC 8017, 2016.
https://www.rfc-editor.org/rfc/rfc8017

[2] Josefsson, S., Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.
https://www.rfc-editor.org/rfc/rfc8032

[3] Langley, A., Hamburg, M., Turner, S. *Elliptic Curves for Security*. RFC 7748, 2016.
https://www.rfc-editor.org/rfc/rfc7748

[4] Barker, E. *Recommendation for Key Management: Part 1 – General*. NIST Special Publication 800-57 Part 1, Revision 5, 2020.
https://doi.org/10.6028/NIST.SP.800-57pt1r5

[5] The Python Cryptographic Authority. *Ed25519 signing*. Cryptography Documentation.
https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/

[6] Pornin, T. *Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)*. RFC 6979, 2013.
https://www.rfc-editor.org/rfc/rfc6979

[7] Barker, E., Chen, L., Vassilev, A., Davis, R., Simon, S., Roginsky, A. *Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography.* NIST Special Publication 800-56B Revision 2, 2019.
`https://csrc.nist.gov/pubs/sp/800/56/b/r2/final`

[8] National Institute of Standards and Technology. *Security Requirements for Cryptographic Modules.* FIPS PUB 140-3, 2019.
`https://csrc.nist.gov/pubs/fips/140-3/final`